

Procedimiento de la red neuronal

Definición, funcionamiento y aprendizaje del perceptrón multicapa del proyecto
`red_neuronal`

Marlon E. Figueroa

Octubre de 2026

Índice

1. Introducción	2
2. Arquitectura del proyecto	3
3. La neurona artificial	3
3.1. Modelo matemático	3
3.2. Interpretación geométrica y necesidad de varias capas	4
4. Funciones de activación	4
5. Capas	5
5.1. Capa densa	6
5.2. Inicialización de los pesos	6
5.3. Dropout	6
6. Definición de la red	7
7. Propagación hacia adelante	8
8. Funciones de pérdida	8
8.1. Error cuadrático medio (regresión)	8
8.2. Entropía cruzada binaria (dos clases)	8
8.3. Softmax y entropía cruzada (varias clases)	9
9. Retropropagación	10
9.1. La regla de la cadena	10
9.2. Gradientes de la capa densa	10
9.3. Gradientes de las activaciones	11
9.4. Grafo de cómputo de una capa	11
10. Optimizadores	12
10.1. Descenso de gradiente estocástico (SGD)	12
10.2. SGD con momento	12
10.3. Adam	12

10.4. Regularización L2	12
11. Proceso de entrenamiento	13
11.1. Épocas y mini-lotes	13
11.2. Algoritmo	13
11.3. Parada temprana	15
12. Ejemplo numérico paso a paso	15
13. Evaluación y generalización	16
13.1. Métricas	16
13.2. Entrenamiento, validación y prueba	16
13.3. Estandarización	16
14. Verificación de los gradientes	16
15. Persistencia del modelo	17
16. Resumen	17

1. Introducción

Este documento describe **cómo está definida, cómo funciona y cómo aprende** la red neuronal implementada en el proyecto `red_neuronal`. La implementación está hecha desde cero con NumPy, por lo que cada concepto matemático de este documento tiene una correspondencia directa con una clase o función del código.

La red es un **perceptrón multicapa** (MLP, *multilayer perceptron*): una sucesión de capas densas, cada una seguida de una función de activación no lineal. Se entrena con **descenso de gradiente por mini-lotes**, y los gradientes se obtienen mediante **retropropagación** (*backpropagation*).

Notación usada a lo largo del documento:

Símbolo	Significado
N	Número de ejemplos en un lote
d	Número de características de entrada
$\mathbf{x} \in \mathbb{R}^d$	Un ejemplo de entrada (vector fila)
$X \in \mathbb{R}^{N \times d}$	Lote de entradas, un ejemplo por fila
$W^{(l)}, \mathbf{b}^{(l)}$	Pesos y sesgos de la capa densa l
$Z^{(l)}$	Salida lineal (pre-activación) de la capa l
$A^{(l)}$	Salida de la activación de la capa l , con $A^{(0)} = X$
φ	Función de activación
$\mathcal{L}$	Función de pérdida
η	Tasa de aprendizaje
θ	Cualquier parámetro entrenable (W o $\mathbf{b}$)

2. Arquitectura del proyecto

El código está dividido en módulos con responsabilidades separadas. La clase `RedNeuronal` coordina a los demás: contiene una lista de capas, una función de pérdida y un optimizador.

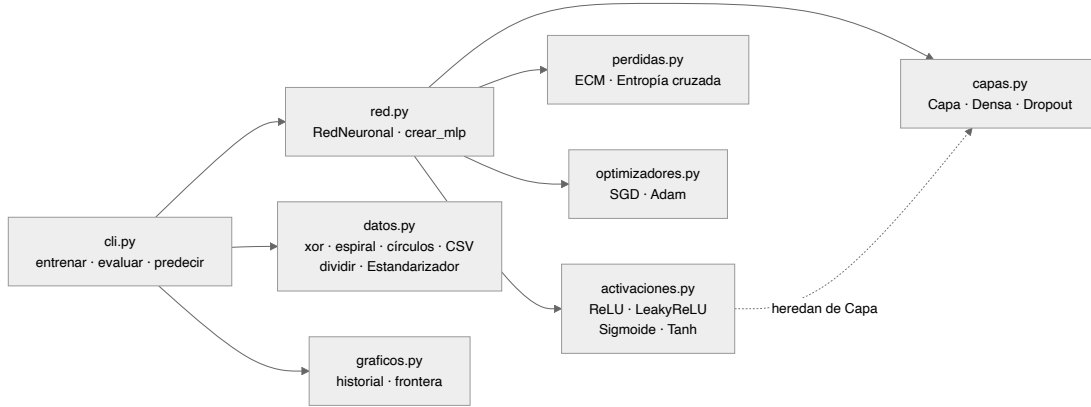


Figura 1: Módulos del paquete y sus dependencias.

Módulo	Contenido	Sección
capas.py	Interfaz Capa, capa Densa, Dropout	Capas
activaciones.py	ReLU, LeakyReLU, Sigmoid, Tanh	Activaciones
perdidas.py	Error cuadrático medio y entropías cruzadas	Pérdidas
optimizadores.py	SGD (con momento) y Adam	Optimizadores
red.py	RedNeuronal y crear_mlp	Entrenamiento
datos.py	Conjuntos de datos y preprocesamiento	Evaluación

3. La neurona artificial

3.1. Modelo matemático

La neurona es la unidad básica de cálculo. Recibe un vector de entradas $\mathbf{x} = (x_1, \dots, x_d)$, multiplica cada entrada por un **peso** w_i , suma los resultados junto con un **sesgo** b y aplica una **función de activación** φ :

$$z = \sum_{i=1}^d w_i x_i + b = \mathbf{w} \cdot \mathbf{x} + b, \quad a = \varphi(z).$$

- Los **pesos** w_i indican cuánto influye cada entrada y en qué sentido (positivo o negativo).
- El **sesgo** b desplaza el umbral a partir del cual la neurona se activa.
- La **activación** φ introduce no linealidad; sin ella, la neurona sería una simple función lineal.

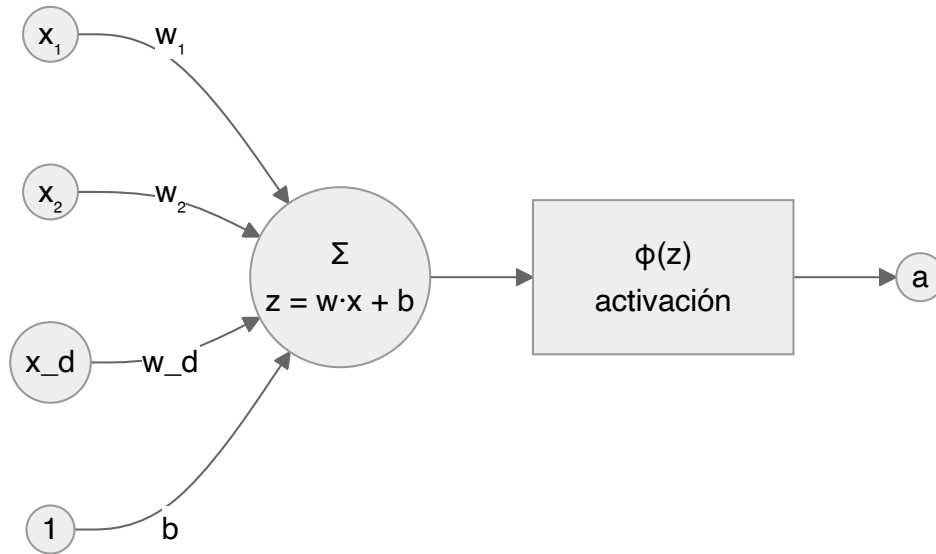


Figura 2: Una neurona: suma ponderada de las entradas más el sesgo, seguida de una activación.

Los pesos y el sesgo son los **parámetros** de la neurona: son los valores que la red ajusta durante el entrenamiento. Aprender consiste exactamente en encontrar valores de $\mathbf{w}$ y b que hagan que la salida a se parezca a la salida deseada.

3.2. Interpretación geométrica y necesidad de varias capas

La ecuación $\mathbf{w} \cdot \mathbf{x} + b = 0$ define un **hiperplano** (una recta en dos dimensiones). Una sola neurona solo puede separar las clases con esa frontera lineal. El ejemplo clásico de su limitación es **XOR**:

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Ninguna recta separa los puntos con salida 1 de los puntos con salida 0. Al apilar neuronas en capas con activaciones no lineales, cada capa transforma el espacio de entrada y la red puede formar fronteras curvas arbitrarias. Por eso el proyecto resuelve XOR con una capa oculta de 8 neuronas, y las espirales con dos capas ocultas de 64.

4. Funciones de activación

Las activaciones se implementan como capas sin parámetros en `activaciones.py`. Se aplican **elemento a elemento**, así que la derivada que se necesita en la retropropagación es simplemente $\varphi'(z)$ en cada posición.

Nombre	$\varphi(z)$	$\varphi'(z)$	Clase
ReLU	$\max(0, z)$	1 si $z > 0$, si no 0	ReLU
Leaky ReLU	z si $z > 0$, si no αz	1 si $z > 0$, si no α	LeakyReLU
Sigmoide	$\sigma(z) = \frac{1}{1 + e^{-z}}$	$\sigma(z) (1 - \sigma(z))$	Sigmoide
Tangente hiperbólica	$\tanh(z)$	$1 - \tanh^2(z)$	Tanh

Observaciones:

- **ReLU** es la activación por defecto. Es barata de calcular y no se satura para valores positivos, lo que mantiene gradientes útiles en redes profundas.
- **Leaky ReLU** usa una pendiente pequeña α (por defecto 0.01) para $z < 0$, de modo que las neuronas nunca quedan con gradiente exactamente cero.
- **Sigmoide** y **Tanh** se saturan: para $|z|$ grande su derivada es casi cero. Funcionan bien en redes pequeñas, como la de XOR.
- La sigmoide se calcula con la identidad $\sigma(z) = \frac{1}{2} (1 + \tanh(\frac{z}{2}))$, que es equivalente pero evita el desbordamiento de e^{-z} cuando z es muy negativo.

Las activaciones guardan en **adelante** lo necesario para la derivada (la máscara $z > 0$ en ReLU, la salida en Sigmoide y Tanh), y lo reutilizan en **atras**:

```
class Sigmoide(Capa):
    def adelante(self, x, entrenando=False):
        self._salida = sigmoide(x)
        return self._salida

    def atras(self, grad):
        return grad * self._salida * (1.0 - self._salida)
```

5. Capas

Todas las capas heredan de la clase **Capa**, que define un contrato de cuatro métodos:

Método	Responsabilidad
<code>adelante(x, entrenando)</code>	Calcula la salida de la capa y guarda lo necesario para el retroceso
<code>atras(grad)</code>	Recibe $\partial\mathcal{L}/\partial\text{salida}$, calcula los gradientes de sus parámetros y devuelve $\partial\mathcal{L}/\partial\text{entrada}$
<code>parametros()</code>	Devuelve pares (parámetro, gradiente) para el optimizador
<code>config()</code>	Devuelve los argumentos necesarios para reconstruir la capa al cargar un modelo

Gracias a este contrato, la red no necesita saber qué tipo de capa está usando: simplemente encadena **adelante** en orden y **atras** en orden inverso.

5.1. Capa densa

Una capa densa (**Densa**) es un conjunto de m neuronas que reciben las mismas n entradas. En lugar de calcular neurona por neurona, se agrupan los pesos en una matriz y se procesa un lote completo de N ejemplos con una sola multiplicación:

$$Z = XW + \mathbf{b}, \quad X \in \mathbb{R}^{N \times n}, W \in \mathbb{R}^{n \times m}, \mathbf{b} \in \mathbb{R}^m, Z \in \mathbb{R}^{N \times m}.$$

La columna j de W contiene los pesos de la neurona j , y el vector $\mathbf{b}$ se suma a cada fila (NumPy lo hace automáticamente mediante *broadcasting*). La capa tiene $n \cdot m + m$ parámetros.

```
class Densa(Capa):
    def adelante(self, x, entrenando=False):
        self._x = x
        return x @ self.W + self.b

    def atras(self, grad):
        self.dW[...] = self._x.T @ grad
        self.db[...] = grad.sum(axis=0)
        return grad @ self.W.T
```

5.2. Inicialización de los pesos

Si todos los pesos empezaran iguales, todas las neuronas de una capa calcularían lo mismo y recibirían el mismo gradiente: nunca se diferenciarían. Por eso los pesos se inicializan **al azar** con una distribución normal de media cero, y los sesgos en cero. La desviación estándar se elige para que la varianza de las activaciones se mantenga estable de capa en capa:

$$\text{He: } W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n}\right) \quad \text{Xavier: } W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n+m}\right)$$

- **He** compensa que ReLU anula la mitad de los valores; se usa con `relu` y `leaky_relu`.
- **Xavier** (Glorot) es adecuada para activaciones simétricas como `tanh` y `sigmoide`, y para la capa de salida.

5.3. Dropout

Dropout es una técnica de regularización. Durante el entrenamiento apaga cada neurona con probabilidad p (la *tasa*), lo que obliga a la red a no depender de neuronas concretas. Se usa la variante **invertida**: los valores que sobreviven se escalan por $1/(1-p)$ para que el valor esperado no cambie.

$$M_{ij} \sim \text{Bernoulli}(1-p), \quad \text{salida} = \frac{X \odot M}{1-p}, \quad \frac{\partial \mathcal{L}}{\partial X} = \frac{\partial \mathcal{L}}{\partial \text{salida}} \odot \frac{M}{1-p}.$$

Al evaluar o predecir (`entrenando=False`), la capa no hace nada. Gracias al escalado invertido, no hace falta corregir nada en ese momento.

6. Definición de la red

La función `crear_mlp` construye la red a partir de una lista de tamaños de capas ocultas, siguiendo siempre el patrón:

$$\underbrace{[\text{Densa} \rightarrow \text{activación} \rightarrow (\text{Dropout})]}_{\text{una vez por capa oculta}} \rightarrow \text{Densa de salida.}$$

La última capa densa **no tiene activación**: produce valores reales llamados *logits*. La transformación final (softmax, sigmoide o identidad) la aplica la función de pérdida, por razones de estabilidad numérica que se explican en la sección de [pérdidas](#).

La configuración por defecto de la CLI para las espirales es $2 \rightarrow 64 \rightarrow 64 \rightarrow 3$:

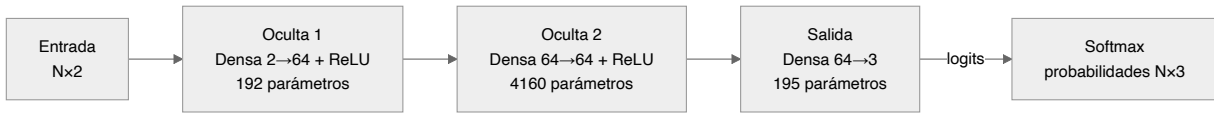


Figura 3: Red por defecto para las espirales: dos capas ocultas de 64 neuronas y 3 salidas.

El número total de parámetros es la suma de los de cada capa densa:

$$(2 \cdot 64 + 64) + (64 \cdot 64 + 64) + (64 \cdot 3 + 3) = 192 + 4160 + 195 = 4547.$$

A nivel de neuronas, cada neurona de una capa está conectada con **todas** las neuronas de la capa anterior. La siguiente figura muestra una versión reducida $2 \rightarrow 3 \rightarrow 2$ para que las conexiones sean visibles:

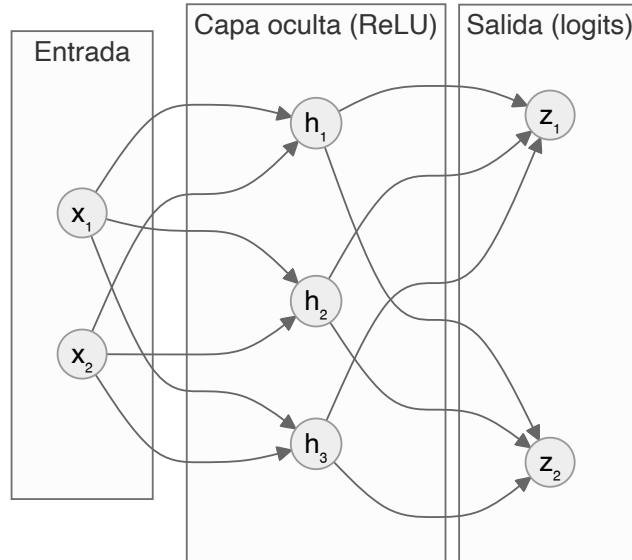


Figura 4: Conexiones completas en una red reducida 2-3-2. Cada flecha es un peso.

7. Propagación hacia adelante

Para obtener una predicción, la entrada atraviesa las capas en orden. Para una red con L capas densas:

$$\begin{aligned}A^{(0)} &= X, \\Z^{(l)} &= A^{(l-1)}W^{(l)} + \mathbf{b}^{(l)}, \quad l = 1, \dots, L, \\A^{(l)} &= \varphi(Z^{(l)}), \quad l = 1, \dots, L-1, \\\hat{Y} &= g(Z^{(L)}),\end{aligned}$$

donde g es la transformación de salida que aplica la pérdida (softmax en clasificación multiclase). En el código, esto es un simple recorrido:

```
def adelante(self, X, entrenando=False):
    for capa in self.capas:
        X = capa.adelante(X, entrenando)
    return X
```

8. Funciones de pérdida

La pérdida $\mathcal{L}$ mide qué tan lejos están las predicciones de los valores correctos. Es el número que el entrenamiento intenta **minimizar**. Cada pérdida de `perdidas.py` implementa:

- `calcular(salida, y)`: el valor de $\mathcal{L}$, promediado sobre el lote.
- `gradiente(salida, y)`: $\partial\mathcal{L}/\partial Z^{(L)}$, el punto de partida de la retropropagación.
- `activar(salida)`: la transformación g que convierte logits en predicciones.

8.1. Error cuadrático medio (regresión)

Para predecir valores continuos, con k salidas por ejemplo:

$$\mathcal{L}_{\text{ECM}} = \frac{1}{Nk} \sum_{i=1}^N \sum_{j=1}^k (\hat{y}_{ij} - y_{ij})^2, \quad \frac{\partial \mathcal{L}}{\partial \hat{y}_{ij}} = \frac{2}{Nk} (\hat{y}_{ij} - y_{ij}).$$

Aquí g es la identidad: la salida de la red es directamente la predicción.

8.2. Entropía cruzada binaria (dos clases)

Con una sola neurona de salida z y objetivo $y \in \{0, 1\}$, la probabilidad predicha es $p = \sigma(z)$ y la pérdida es

$$\mathcal{L}_{\text{ECB}} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right].$$

Calcular primero p y luego $\log p$ puede dar $\log 0$. Por eso se trabaja directamente con el logit, usando la forma equivalente y estable:

$$\ell(z, y) = \max(z, 0) - z y + \log(1 + e^{-|z|}), \quad \frac{\partial \mathcal{L}}{\partial z_i} = \frac{\sigma(z_i) - y_i}{N}.$$

8.3. Softmax y entropía cruzada (varias clases)

Para C clases, la capa de salida produce C logits por ejemplo. La función **softmax** los convierte en probabilidades que suman 1:

$$p_{ic} = \text{softmax}(\mathbf{z}_i)_c = \frac{e^{z_{ic}}}{\sum_{k=1}^C e^{z_{ik}}}.$$

Si c_i es la clase correcta del ejemplo i , la entropía cruzada penaliza que la probabilidad asignada a esa clase sea baja:

$$\mathcal{L}_{\text{EC}} = -\frac{1}{N} \sum_{i=1}^N \log p_{ic_i}.$$

Para evitar desbordamientos se calcula el **logaritmo de softmax** restando antes el máximo de cada fila, lo que no cambia el resultado:

$$\log p_{ic} = (z_{ic} - m_i) - \log \sum_{k=1}^C e^{z_{ik} - m_i}, \quad m_i = \max_k z_{ik}.$$

La gran ventaja de combinar softmax con entropía cruzada es que el gradiente respecto a los logits es extremadamente simple. Con $\mathbf{y}_i$ el vector *one-hot* de la clase correcta:

$$\frac{\partial \mathcal{L}}{\partial z_{ic}} = \frac{p_{ic} - y_{ic}}{N}.$$

Derivación. Para un ejemplo, $\ell = -\log p_{c^*} = -z_{c^*} + \log \sum_k e^{z_k}$. Derivando respecto a z_c : el primer término aporta -1 solo si $c = c^*$, y el segundo aporta $e^{z_c} / \sum_k e^{z_k} = p_c$. Por tanto $\partial \ell / \partial z_c = p_c - y_c$.

Es decir: **probabilidad predicha menos valor correcto**. Si la red asigna probabilidad 0.9 a la clase correcta, el gradiente en esa posición es -0.1 (empuja a subirla un poco); si asigna 0.3 a una clase incorrecta, el gradiente es $+0.3$ (empuja a bajarla).

Tarea (salidas)	Pérdida	g	$\partial \mathcal{L} / \partial z$
Regresión (k)	ErrorCuadraticoMedio	identidad	$\frac{2}{Nk}(\hat{y} - y)$
Binaria (1)	EntropiaCruzadaBinaria	sigmoide	$\frac{1}{N}(\sigma(z) - y)$
Multiclase (C)	EntropiaCruzada	softmax	$\frac{1}{N}(p - y)$

9. Retropropagación

9.1. La regla de la cadena

Para mejorar la red hay que saber cómo cambia la pérdida cuando cambia cada parámetro, es decir, el **gradiente** $\partial\mathcal{L}/\partial\theta$. La red es una composición de funciones, así que el gradiente se obtiene con la **regla de la cadena**. Por ejemplo, para los pesos de la última capa:

$$\frac{\partial\mathcal{L}}{\partial W^{(L)}} = \frac{\partial\mathcal{L}}{\partial Z^{(L)}} \cdot \frac{\partial Z^{(L)}}{\partial W^{(L)}}.$$

Y para una capa anterior, la cadena se alarga atravesando las capas posteriores. La retropropagación organiza este cálculo de forma eficiente: recorre la red **de la salida hacia la entrada**, y cada capa recibe el gradiente respecto a su salida, calcula el de sus parámetros y pasa a la capa anterior el gradiente respecto a su entrada. Así, cada derivada intermedia se calcula una sola vez.

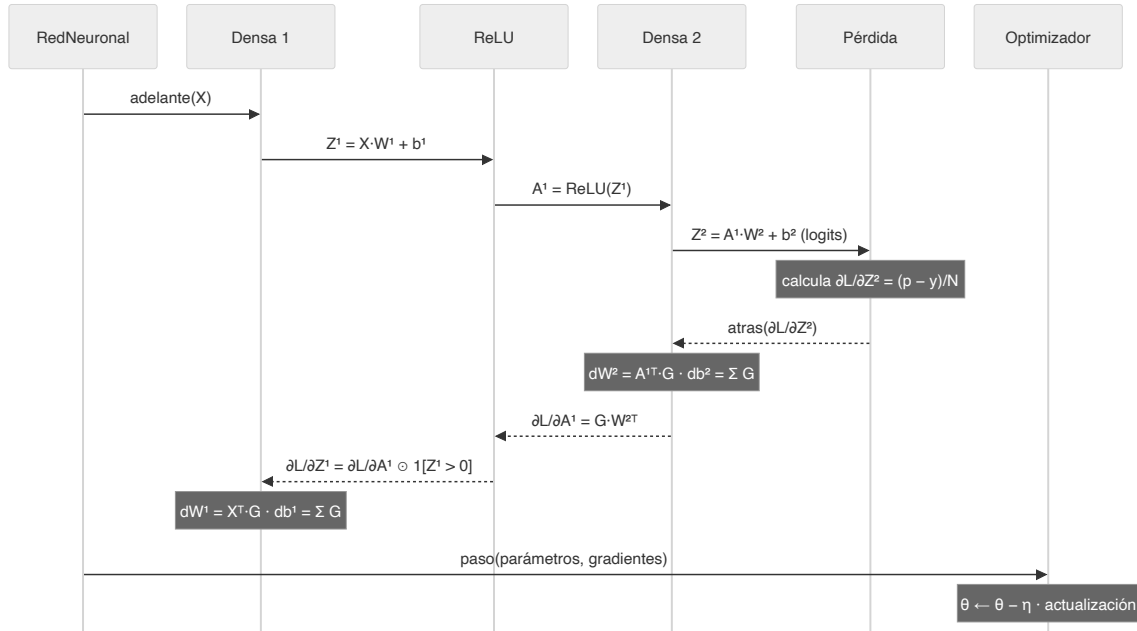


Figura 5: Un paso de entrenamiento: propagación hacia adelante, retropropagación y actualización.

9.2. Gradientes de la capa densa

Sea $G = \partial\mathcal{L}/\partial Z$ el gradiente que llega a una capa densa con $Z = XW + \mathbf{b}$. Como $Z_{ij} = \sum_k X_{ik} W_{kj} + b_j$:

- Cada peso W_{kj} afecta a la columna j de Z en todos los ejemplos, con coeficiente X_{ik} . Sumando sobre el lote: $\frac{\partial\mathcal{L}}{\partial W_{kj}} = \sum_i X_{ik} G_{ij}$.
- Cada sesgo b_j se suma a la columna j con coeficiente 1: $\frac{\partial\mathcal{L}}{\partial b_j} = \sum_i G_{ij}$.
- Cada entrada X_{ik} afecta a toda la fila i de Z con coeficientes W_{kj} : $\frac{\partial\mathcal{L}}{\partial X_{ik}} = \sum_j G_{ij} W_{kj}$.

En forma matricial, exactamente lo que implementa `Densa.atras`:

$$\boxed{\frac{\partial \mathcal{L}}{\partial W} = X^\top G, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_{i=1}^N G_{i,:}, \quad \frac{\partial \mathcal{L}}{\partial X} = G W^\top}$$

Las dimensiones coinciden: $X^\top G$ es $(n \times N)(N \times m) = n \times m$, igual que W ; y $GW^\top$ es $(N \times m)(m \times n) = N \times n$, igual que X .

Como las pérdidas ya dividen su gradiente entre N , $\mathbf{dW}$ y $\mathbf{db}$ son gradientes **promedio** del lote, y la escala de la tasa de aprendizaje no depende del tamaño del lote.

9.3. Gradientes de las activaciones

Para una activación elemento a elemento $A = \varphi(Z)$:

$$\frac{\partial \mathcal{L}}{\partial Z} = \frac{\partial \mathcal{L}}{\partial A} \odot \varphi'(Z),$$

donde $\odot$ es el producto elemento a elemento. En ReLU esto equivale a dejar pasar el gradiente donde $Z > 0$ y anularlo en el resto.

9.4. Grafo de cómputo de una capa

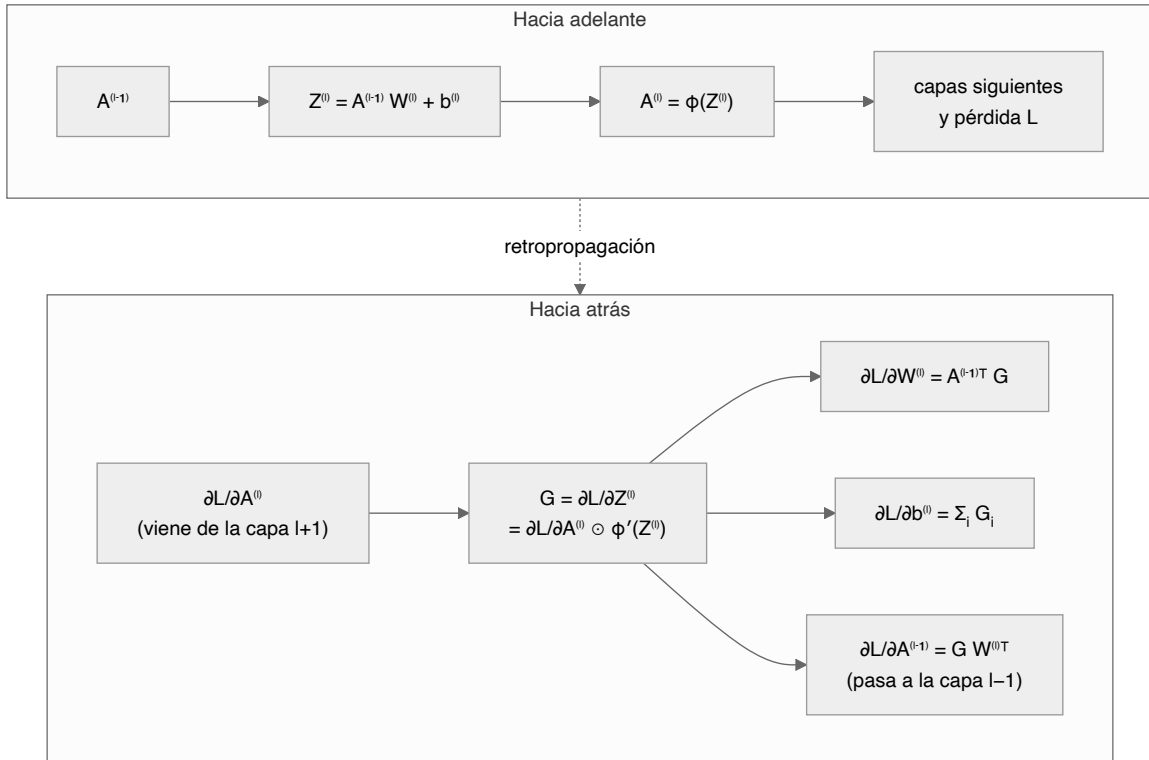


Figura 6: Grafo de cómputo de una capa oculta l . Arriba, el cálculo hacia adelante; abajo, el flujo del gradiente, que llega desde las capas siguientes.

10. Optimizadores

Una vez calculados los gradientes, el optimizador actualiza los parámetros. El gradiente apunta en la dirección en que la pérdida **crece** más rápido, así que se avanza en sentido contrario.

10.1. Descenso de gradiente estocástico (SGD)

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}.$$

Se llama *estocástico* porque el gradiente se calcula sobre un mini-lote aleatorio y no sobre todos los datos: es una estimación ruidosa pero barata del gradiente real. La tasa de aprendizaje η controla el tamaño del paso. Si es muy grande, la pérdida oscila o diverge; si es muy pequeña, el aprendizaje es lento.

10.2. SGD con momento

El momento acumula una “velocidad” $\mathbf{v}$ que promedia los gradientes recientes. Así se suavizan las oscilaciones y se acelera el avance en direcciones consistentes:

$$\mathbf{v} \leftarrow \mu \mathbf{v} - \eta \nabla_{\theta} \mathcal{L}, \quad \theta \leftarrow \theta + \mathbf{v},$$

con $\mu \in [0, 1)$ el coeficiente de momento (0.9 por defecto en la CLI).

10.3. Adam

Adam (*adaptive moment estimation*) mantiene, para cada parámetro, una media móvil del gradiente $\mathbf{m}$ y de su cuadrado $\mathbf{v}$. Con $\mathbf{g}_t$ el gradiente en el paso t :

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, & \hat{\mathbf{m}}_t &= \frac{\mathbf{m}_t}{1 - \beta_1^t}, \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, & \hat{\mathbf{v}}_t &= \frac{\mathbf{v}_t}{1 - \beta_2^t}, \\ \theta_t &= \theta_{t-1} - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \end{aligned}$$

- $\hat{\mathbf{m}}_t$ y $\hat{\mathbf{v}}_t$ son las **correcciones de sesgo**: como $\mathbf{m}$ y $\mathbf{v}$ empiezan en cero, sin corrección serían demasiado pequeños en los primeros pasos.
- Dividir por $\sqrt{\hat{\mathbf{v}}_t}$ da a cada parámetro su propio tamaño de paso: los parámetros con gradientes grandes o ruidosos avanzan con más cuidado.
- Valores por defecto: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. La CLI usa $\eta = 0.01$.

10.4. Regularización L2

Con `decaimiento_peso` = $\lambda > 0$, se suma $\lambda \theta$ al gradiente antes de actualizar. Equivale a añadir $\frac{\lambda}{2} \|\theta\|^2$ a la pérdida, lo que favorece pesos pequeños y reduce el sobreajuste:

$$\mathbf{g} \leftarrow \nabla_{\theta} \mathcal{L} + \lambda \theta.$$

11. Proceso de entrenamiento

11.1. Épocas y mini-lotes

- Un **mini-lote** es un subconjunto de `tam_lote` ejemplos (32 por defecto). Con cada mini-lote se da un **paso** de optimización.
- Una **época** es una pasada completa por todos los datos de entrenamiento. Al inicio de cada época los ejemplos se **barajan**, para que los lotes sean distintos y el gradiente no tenga sesgos por el orden de los datos.

Con 480 ejemplos y lotes de 32, cada época tiene $\lceil 480/32 \rceil = 15$ pasos.

11.2. Algoritmo

El método `RedNeuronal.entrenar` implementa el siguiente algoritmo:

1. Para cada época $e = 1, \dots, E$:
 - a. Barajar los índices de los ejemplos.
 - b. Para cada mini-lote (X_B, Y_B) :
 - i. **Adelante:** $Z^{(L)} = \text{red}(X_B)$, con dropout activo.
 - ii. **Gradiente de la pérdida:** $G = \partial \mathcal{L}(Z^{(L)}, Y_B) / \partial Z^{(L)}$.
 - iii. **Atrás:** propagar G por las capas en orden inverso, llenando dW y db .
 - iv. **Actualizar:** el optimizador modifica cada θ con su gradiente.
 - c. Evaluar pérdida y precisión en entrenamiento y validación (sin dropout).
 - d. Si hay **paciencia**, comprobar la parada temprana.
2. Si hubo parada temprana, restaurar los mejores pesos.
3. Devolver el historial de métricas.

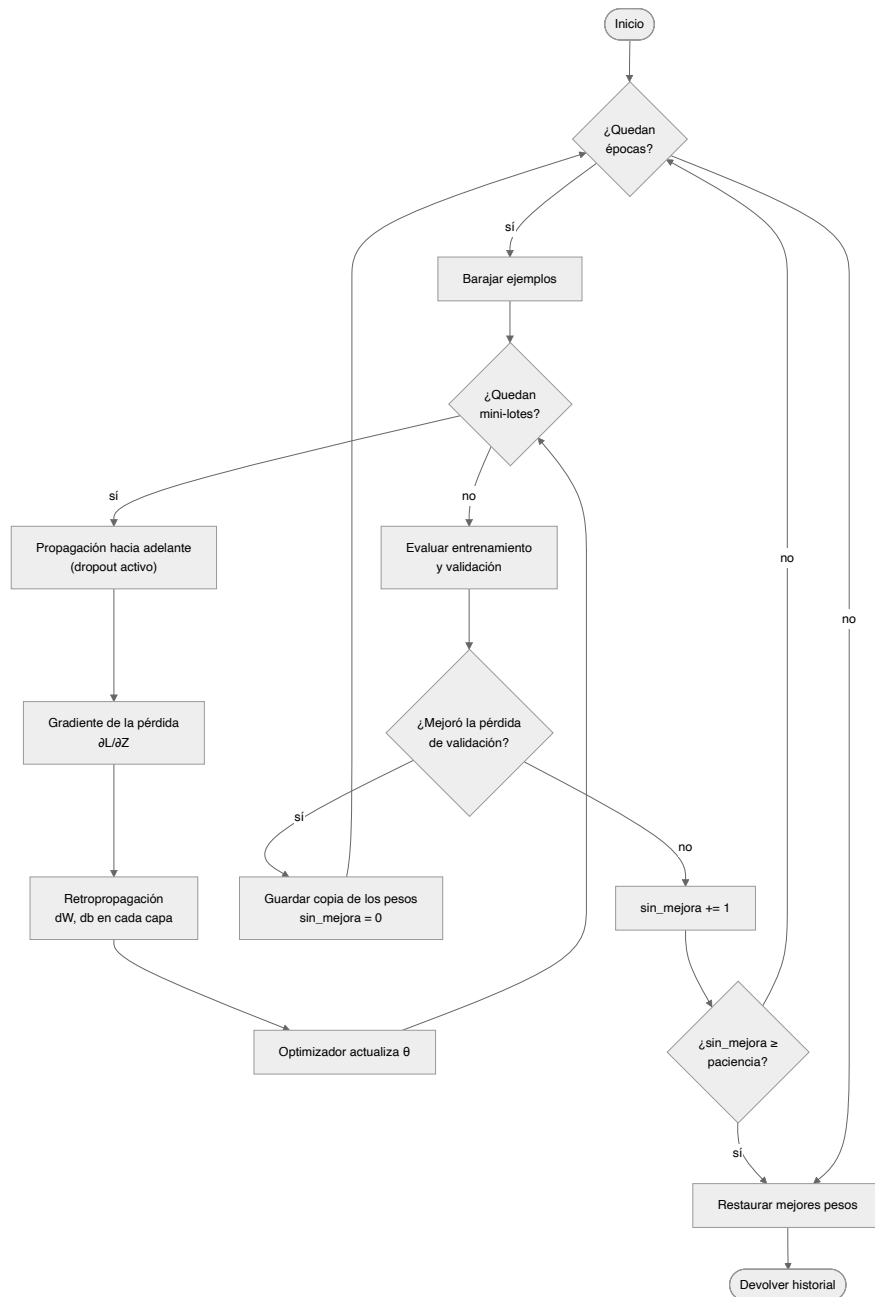


Figura 7: Flujo completo del entrenamiento, con parada temprana opcional.

El núcleo del bucle en el código es:

```

for inicio in range(0, len(X), tam_lote):
    lote = indices[inicio : inicio + tam_lote]
    salida = self.adelante(X[lote], entrenando=True)
    self.atras(self.perdida.gradiente(salida, y[lote]))
    self.optimizador.paso(self.parametros())
  
```

11.3. Parada temprana

Si se indica **paciencia** y hay datos de validación, la red guarda una copia de los pesos cada vez que la pérdida de validación alcanza un nuevo mínimo. Si pasan **paciencia** épocas sin mejorar, el entrenamiento se detiene y se restauran los mejores pesos. Así se evita seguir entrenando cuando la red ya empezó a sobreajustar.

12. Ejemplo numérico paso a paso

Para ver el aprendizaje en acción, se sigue un paso de entrenamiento de **una sola neurona** con activación sigmoide y entropía cruzada binaria, con un único ejemplo.

Datos iniciales: $\mathbf{x} = (1, 2)$, objetivo $y = 1$, pesos $\mathbf{w} = (0.5, -0.25)$, sesgo $b = 0.1$ y tasa de aprendizaje $\eta = 0.1$.

1. Propagación hacia adelante.

$$z = 0.5 \cdot 1 + (-0.25) \cdot 2 + 0.1 = 0.1, \quad p = \sigma(0.1) = 0.52498.$$

La neurona asigna una probabilidad de 52.5 % a la clase correcta: casi un volado.

2. Pérdida.

$$\mathcal{L} = -\log(0.52498) = 0.64440.$$

3. Retropropagación. Con softmax/sigmoide y entropía cruzada, el gradiente respecto al logit es probabilidad menos objetivo:

$$\frac{\partial \mathcal{L}}{\partial z} = p - y = 0.52498 - 1 = -0.47502.$$

Por la regla de la cadena, como $z = w_1 x_1 + w_2 x_2 + b$:

$$\frac{\partial \mathcal{L}}{\partial w_1} = -0.47502 \cdot 1 = -0.47502, \quad \frac{\partial \mathcal{L}}{\partial w_2} = -0.47502 \cdot 2 = -0.95004, \quad \frac{\partial \mathcal{L}}{\partial b} = -0.47502.$$

Los gradientes son negativos: aumentar estos parámetros **reduce** la pérdida.

4. Actualización (SGD).

$$\begin{aligned} w_1 &\leftarrow 0.5 - 0.1 \cdot (-0.47502) = 0.54750, \\ w_2 &\leftarrow -0.25 - 0.1 \cdot (-0.95004) = -0.15500, \\ b &\leftarrow 0.1 - 0.1 \cdot (-0.47502) = 0.14750. \end{aligned}$$

5. Comprobación. Con los nuevos parámetros:

$$z = 0.5475 - 0.31 + 0.1475 = 0.38501, \quad p = \sigma(0.38501) = 0.59508, \quad \mathcal{L} = 0.51906.$$

Tras un solo paso, la probabilidad de la clase correcta subió de 52.5 % a 59.5 % y la pérdida bajó de 0.644 a 0.519. Entrenar una red consiste en repetir este mismo proceso miles de veces, con miles de parámetros a la vez.

13. Evaluación y generalización

13.1. Métricas

- **Pérdida:** el valor de $\mathcal{L}$ sobre un conjunto de datos.
- **Precisión** (*accuracy*), solo en clasificación: fracción de ejemplos cuya clase predicha coincide con la correcta. La clase predicha es la de mayor logit, que es también la de mayor probabilidad:

$$\text{precisión} = \frac{1}{N} \sum_{i=1}^N \mathbb{1} \left[\arg \max_c z_{ic} = c_i \right].$$

13.2. Entrenamiento, validación y prueba

Una red puede memorizar los datos de entrenamiento sin aprender el patrón general. Para detectarlo, `datos.dividir` separa los datos (80 % / 20 % por defecto):

- **Entrenamiento:** los únicos datos con los que se calculan gradientes.
- **Validación / prueba:** datos que la red nunca usa para aprender; miden si **generaliza**.

Si la pérdida de entrenamiento sigue bajando mientras la de validación sube, la red está **sobreajustando**. Las herramientas del proyecto para combatirlo son el dropout, la regularización L2 y la parada temprana.

13.3. Estandarización

Cuando las características tienen escalas muy distintas (por ejemplo, metros y milímetros), el entrenamiento se vuelve lento e inestable. Con datos CSV, la CLI aplica el **Estandarizador**, ajustado **solo** con los datos de entrenamiento:

$$x'_j = \frac{x_j - \mu_j}{s_j},$$

donde μ_j y s_j son la media y la desviación estándar de la característica j . Esos valores se guardan dentro del modelo para aplicar la misma transformación al predecir.

14. Verificación de los gradientes

Un error en la retropropagación no produce excepciones: la red simplemente aprende mal. Por eso `tests/test_gradientes.py` compara los gradientes analíticos con una aproximación numérica por **diferencias centrales**:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} \approx \frac{\mathcal{L}(\theta_k + \varepsilon) - \mathcal{L}(\theta_k - \varepsilon)}{2\varepsilon}, \quad \varepsilon = 10^{-6}.$$

La prueba construye una red con capas `Densa`, `Tanh`, `LeakyReLU` y `Sigmoide`, y verifica cada parámetro con las tres funciones de pérdida. Ambos métodos deben coincidir con un error relativo menor que 10^{-4} . Cualquier capa nueva puede validarse de la misma forma.

15. Persistencia del modelo

`RedNeuronal.guardar` escribe un archivo `.npz` (formato comprimible de NumPy) que contiene:

- **arquitectura**: un JSON con el tipo y la configuración (`config()`) de cada capa, el nombre de la pérdida y los metadatos (tarea, nombres de clases y parámetros de estandarización).
- **p0, p1, ...**: los arreglos de pesos y sesgos, en el mismo orden que `parametros()`.

`RedNeuronal.cargar` reconstruye las capas a partir del JSON y copia los pesos. El archivo se lee con `allow_pickle=False`, de modo que cargar un modelo no ejecuta código arbitrario.

16. Resumen

Concepto	Fórmula clave	Implementación
Neurona	$a = \varphi(\mathbf{w} \cdot \mathbf{x} + b)$	Columna de <code>Densa.W</code>
Capa densa	$Z = XW + \mathbf{b}$	<code>Densa.adelante</code>
Activación	$A = \varphi(Z)$	<code>activaciones.py</code>
Pérdida	$\mathcal{L} = -\frac{1}{N} \sum_i \log p_{ic_i}$	<code>EntropiaCruzada</code>
Gradiente de salida	$\partial \mathcal{L} / \partial Z = (P - Y) / N$	<code>Perdida.gradiente</code>
Retropropagación	$\partial \mathcal{L} / \partial W = X^\top G,$ $\partial \mathcal{L} / \partial X = G W^\top$	<code>Densa.atras</code>
Actualización	$\theta \leftarrow \theta - \eta \mathbf{g}$	SGD, Adam
Entrenamiento	épocas $\times$ mini-lotes	<code>RedNeuronal.entrenar</code>

En una frase: **la red transforma las entradas capa por capa, mide su error con la pérdida, calcula con la regla de la cadena cuánto contribuyó cada peso a ese error y ajusta cada peso un poco en la dirección que lo reduce**, repitiendo el proceso hasta que las predicciones son buenas también en datos que nunca vio.